Name:Bhange Nikita

Roll No. : 2501019

DSA_ Handson

Q.1 Singly Linked List

Write a program to perform the following operations on a Singly Linked List:

```
class newNode:
    def __init__(self,data):
        self.data = data
        self.next  = None


class LinkedList():
    def __init__(self):
        self.head = None


    def append(self,data):
        newnode = newNode(data)
        if self.head is None:
            self.head = newnode
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = newnode
```

1.Append a node at the end of the list.

```
    def appendatend(self,data):
        newnode = newNode(data)
        temp = self.head
        while temp:
            temp = temp.next
```

```
    temp.next = newnode
```

2. Print the list.

```
def printlist(self):
    temp = self.head
    while(temp):
        print(temp.data,end="->")
        temp = temp.next
    print('None')
```

3. Find the Minimum and Maximum node value.

```
def findminmax(self):
    temp = self.head
    min = temp.data
    max= temp.data

    while(temp):
        if temp.data>max:
            max= temp.data
        if temp.data<min:
            min = temp.data
        temp = temp.next
    print("min value is ",min)
    print("max value is ",max)
```

4. Count Even and odd value nodes.

```
def even_odd(self):
    even =0
    odd= 0
    temp = self.head
    while(temp):
        if temp.data % 2 ==0:
            even +=1
```

```
        else :
            odd +=1
        temp = temp.next

    print("count of even node:",even)
    print("count of odd is",odd)
```

5. Insert a node at a specific position.

```
def position(self,position,data):
    newnode = newNode(data)

    temp = self.head
    index = 1
    while temp:
        if index == position:
            break
        temp = temp.next
        index +=1

    newnode.next = temp.next
    temp.next = newnode
```

. 6. Delete the last node.

```
def deletelastnode(self):
    temp = self.head
    if self.head is None:
        print("list is empty")

    while temp.next.next :
        temp = temp.next
```

```
    temp.next=None
```

7. Delete the middle node

```
def deletemid(self,position):
    temp = self.head
    index = 2
    while temp:
        if index == position:
            break
        temp = temp.next
        index +=1
    temp.next = temp.next.next
```

8. Search for an element in the list

```
def search(self,data):
    temp = self.head
    pos =1
    while temp:
        if temp.data == data:
            print("node is at",pos)
        temp = temp.next
        pos+=1
```

. 9. Merge two lists.

```
def merge(self,list2):
    if self.head is None :
        self.head.next = list2.head

    temp = self.head
    while temp.next:
        temp = temp.next
```

```
        temp.next = list2.head
```

10. Reverse the list.

```
    def reverse(self):
        current = self.head
        prev = None
        while current:
            nextnode = current.next
            current.next = prev
            prev = current
            current = nextnode
        self.head = prev
```

11. Swap node values of consecutive nodes

```
    def swap(self):
        temp = self.head
        while temp:
            a = temp.data
            temp.data = temp.next.data
            temp.next.data = a
            temp = temp.next.next


arr1 = [2,5,7,4]
arr2 =[10,20]
list1  = LinkedList()
list2 = LinkedList()
for i in range(len(arr1)):
    list1.append(arr1[i])


for i in range(len(arr2)):
    list2.append(arr2[i])


list1.search(5)
```

```
list1.printlist()

list1.findminmax()

list1.position(2,9)

list1.deletelastnode()

list1.printlist()

list1.merge(list2)

list1.printlist()

print("print mid")

list1.deletemid(3)

list1.printlist()

list1.reverse()

list1.printlist()
```

```
on/singly.py
node is at 2
2->5->7->4->None
min value is  2
max value is  7
2->5->9->7->None
2->5->9->7->10->20->None
print mid
2->5->7->10->20->None
20->10->7->5->2->None
PS C:\Users\DELL\Desktop\python>
```

Doubly Linked List

Write a program to perform the following operations on a Doubly Linked List:

```
class newNode:

    def __init__(self, data):

        self.data = data

        self.prev = None

        self.next = None
```

```
class DoublyLinkedList:
    def __init__(self):
        self.head = None
```

1. Append a node at the end

```
    def append(self, data):
        node = newNode(data)
        if self.head is None:
            self.head = node
            return

        temp = self.head
        while temp.next:
            temp = temp.next

        temp.next = node
        node.prev = temp
```

2. Print the list (Easy)

```
    def print_list(self):
        temp = self.head
        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.next
        print("None")
```

3. Count total nodes (Easy)

```
    def count_nodes(self):
        count = 0
        temp = self.head
```

```
    while temp:
        count += 1
        temp = temp.next
    return count
```

4. Print list in reverse order (Medium)

```
def print_reverse(self):
    temp = self.head
    if temp is None:
        return

    while temp.next:
        temp = temp.next

    while temp:
        print(temp.data, end=" <-> ")
        temp = temp.prev
    print("None")
```

5. Insert at any position (Medium)

```
def insert_at_position(self, data, pos):
    node = newNode(data)

    if pos == 1:
        node.next = self.head
        if self.head:
            self.head.prev = node
        self.head = node
        return
```

```
    temp = self.head
    for _ in range(pos - 2):
        if temp is None:
            return
        temp = temp.next

    node.next = temp.next
    if temp.next:
        temp.next.prev = node
    temp.next = node
    node.prev = temp
```

6. Delete node at specific position (Medium)

```
def delete_at_position(self, pos):
    if self.head is None:
        return

    if pos == 1:
        self.head = self.head.next
        if self.head:
            self.head.prev = None
        return

    temp = self.head
    for _ in range(pos - 1):
        temp = temp.next
        if temp is None:
            return

    if temp.next:
        temp.next.prev = temp.prev
```

```
    if temp.prev:
        temp.prev.next = temp.next
```

7. Search element and print position (Medium)

```
def search(self, key):
    pos = 1
    temp = self.head
    while temp:
        if temp.data == key:
            print(f"Element {key} found at position {pos}")
            return
        pos += 1
        temp = temp.next
    print("Element not found")
```

8. Delete alternate nodes (Hard)

```
def delete_alternate(self):
    temp = self.head
    while temp and temp.next:
        next_node = temp.next.next
        temp.next = next_node
        if next_node:
            next_node.prev = temp
        temp = next_node
```

9. Reverse the list (Hard)

```
def reverse(self):
    temp = None
    current = self.head
```

```
        while current:
            temp = current.prev
            current.prev = current.next
            current.next = temp
            current = current.prev

        if temp:
            self.head = temp.prev
dll = DoublyLinkedList()

dll.append(10)
dll.append(20)
dll.append(30)
dll.append(40)

dll.print_list()
print("Total Nodes:", dll.count_nodes())

dll.print_reverse()

dll.insert_at_position(25, 3)
dll.print_list()

dll.delete_at_position(2)
dll.print_list()

dll.search(30)

dll.delete_alternate()
dll.print_list()
```

dll.reverse()

dll.print_list()

```
10->none
PS C:\Users\DELL\Desktop\python> & C:/Users/DELL/AppData/Local/Pro
rams/Python/Python313/python.exe c:/Users/DELL/Desktop/python/hand
on/doublylinkedlist.py
10->11->12->13->14->15->none
total nodes are: 6
10->11->12->40->13->14->15->none
10->11->40->13->14->15->none
node is founded at 3
PS C:\Users\DELL\Desktop\python>
```

stack:

Write a program to perform the following operations on a Stack:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:
    def __init__(self):
        self.top = None

    # 1. Push element onto stack (Easy)
    def push(self, data):
        node = Node(data)
        node.next = self.top
        self.top = node
        print(f"Pushed {data}")
```

2. Pop element from stack (Easy)

```
def pop(self):
    if self.is_empty():
        print("Stack is empty")
        return None
    popped = self.top.data
    self.top = self.top.next
    return popped
```

3. Peek top element (Easy)

```
def peek(self):
    if self.is_empty():
        print("Stack is empty")
        return None
    return self.top.data

def is_empty(self):
    return self.top is None

def display(self):
    temp = self.top
    print("Stack (Top → Bottom): ", end="")
    while temp:
        print(temp.data, end=" -> ")
        temp = temp.next
    print("None")
```

4. Reverse the stack using an array.

```
def reverse_stack_using_array(stack):
    arr = []
```

```
    while not stack.is_empty():
        arr.append(stack.pop())

    for item in arr:
        stack.push(item)
```

5. Reverse the stack using another stack.

```
def reverse_stack_using_stack(stack):
    temp_stack = Stack()

    while not stack.is_empty():
        temp_stack.push(stack.pop())

    stack.top = temp_stack.top
```

6. Reverse a string using stack.

```
def reverse_string(s):
    st = Stack()

    for ch in s:
        st.push(ch)

    rev = ""
    while not st.is_empty():
        rev += st.pop()

    return rev
```

7. Check for balanced parenthesis.

```
def is_balanced(expression):
    st = Stack()
```

```
    pairs = {')': '(', '}': '{', ']': '['}

    for ch in expression:
        if ch in "({[":
            st.push(ch)
        elif ch in ")}]":
            if st.is_empty() or st.pop() != pairs[ch]:
                return False

    return st.is_empty()

s = Stack()

s.push(10)
s.push(20)
s.push(30)
s.display()

print("Popped:", s.pop())
print("Top Element:", s.peek())

reverse_stack_using_array(s)
s.display()

reverse_stack_using_stack(s)
s.display()

print("Reversed String:", reverse_string("LINKEDLIST"))

expr = "{[()()]}"
print("Balanced Parenthesis:", is_balanced(expr))
```

Linear Queue

Write a program to perform the following operations on a Linear Queue:

```
class LinearQueue:
    def __init__(self):
        self.queue = []

    # 1. Enqueue an element into the queue
    def enqueue(self, item):
        self.queue.append(item)

    # 2. Dequeue an element from the queue
    def dequeue(self):
        if not self.is_empty():
            return self.queue.pop(0)
        else:
            return "Queue is empty"

    # 3. Peek the front element of the queue
    def peek(self):
        if not self.is_empty():
            return self.queue[0]
        else:
            return "Queue is empty"

    # 4. Reverse the queue using stack
    def reverse(self):
        stack = []
        while self.queue:
```

```
        stack.append(self.queue.pop(0))
    while stack:
        self.queue.append(stack.pop())

# 5. Count occurrences of an element in the queue
def count_occurrences(self, item):
    return self.queue.count(item)

# 6. Search for an element in the queue
def search(self, item):
    return item in self.queue

# 7. Merge two queues into one
def merge(self, other_queue):
    merged_queue = LinearQueue()
    merged_queue.queue = self.queue + other_queue.queue
    return merged_queue

# 8. Check if two queues are identical
def is_identical(self, other_queue):
    return self.queue == other_queue.queue

def is_empty(self):
    return len(self.queue) == 0

def display(self):
    return self.queue

q1 = LinearQueue()
q2 = LinearQueue()
```

```
q1.enqueue(10)
q1.enqueue(20)
q1.enqueue(30)
q2.enqueue(10)
q2.enqueue(20)
q2.enqueue(30)

print("Queue 1:", q1.display())
print("Queue 2:", q2.display())

print("Dequeued from Queue 1:", q1.dequeue())

print("Peek front of Queue 1:", q1.peek())

q1.reverse()
print("Queue 1 after reverse:", q1.display())

# Count occurrences
print("Occurrences of 20 in Queue 1:", q1.count_occurrences(20))

# Search
print("Is 30 in Queue 1?", q1.search(30))

# Merge
merged = q1.merge(q2)
print("Merged Queue:", merged.display())

# Identical check
print("Are Queue 1 and Queue 2 identical?", q1.is_identical(q2))
```

priority queue:

```
class Node:
    def __init__(self, data, priority):
        self.data = data
        self.priority = priority
        self.next = None

class PriorityQueue:
    def __init__(self):
        self.front = None

    # 1. Enqueue with priority (Easy)
    def enqueue(self, data, priority):
        node = Node(data, priority)

        # If queue is empty or higher priority than front
        if self.front is None or priority < self.front.priority:
            node.next = self.front
            self.front = node
            return

        temp = self.front
        while temp.next and temp.next.priority <= priority:
            temp = temp.next

        node.next = temp.next
        temp.next = node

    # 2. Display queue (Easy)
    def display(self):
        temp = self.front
        print("Priority Queue:")
```

```
        while temp:
            print(f"{temp.data} (P:{temp.priority}) -> ", end="")
            temp = temp.next
        print("None")
```

3. Dequeue highest priority element (Medium)

```
    def dequeue(self):
        if self.front is None:
            print("Queue is empty")
            return None

        removed = self.front.data
        self.front = self.front.next
        return removed

pq = PriorityQueue()

pq.enqueue("A", 3)
pq.enqueue("B", 1)
pq.enqueue("C", 2)

pq.display()

print("Dequeued:", pq.dequeue())
pq.display()
```

```
--- Priority Queue ---
Enqueued Task1 with priority 2
Enqueued Task2 with priority 1
Enqueued Task3 with priority 3
Priority Queue: Task2(1) -> Task1(2) -> Task3(3) -> None
Front element is Task2 with priority 1
Dequeued Task2 with priority 1
Priority Queue: Task1(2) -> Task3(3) -> None
PS C:\Users\DELL\Desktop\python>
```

BST:

```
class BSTNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
class BinarySearchTree:
    def __init__(self):
        self.root = None
```

1. Insert node (Easy)

```
    def insert(self, data):
        self.root = self._insert(self.root, data)

    def _insert(self, root, data):
        if root is None:
            return BSTNode(data)
        if data < root.data:
            root.left = self._insert(root.left, data)
        elif data > root.data:
            root.right = self._insert(root.right, data)
        return root
```

2. Find minimum value (Easy)

```
def find_min(self):
    current = self.root
    while current.left:
        current = current.left
    return current.data
```

3. Find maximum value (Easy)

```
def find_max(self):
    current = self.root
    while current.right:
        current = current.right
    return current.data
```

4. Search node and print level (Medium)

```
def search_level(self, key):
    level = 1
    current = self.root
    while current:
        if key == current.data:
            print(f"Element found at level {level}")
            return
        elif key < current.data:
            current = current.left
        else:
            current = current.right
        level += 1
    print("Element not found")
```

5. Count total nodes (Easy)

```
def count_nodes(self, root):
    if root is None:
        return 0
    return 1 + self.count_nodes(root.left) + self.count_nodes(root.right)
```

6. Count nodes in left subtree (Medium)

```
def count_left_subtree(self):
    return self.count_nodes(self.root.left)
```

7. Count nodes in right subtree (Medium)

```
def count_right_subtree(self):
    return self.count_nodes(self.root.right)
```

8. Delete specific node (Medium)

```
def delete(self, key):
    self.root = self._delete(self.root, key)

def _delete(self, root, key):
    if root is None:
        return root

    if key < root.data:
        root.left = self._delete(root.left, key)
    elif key > root.data:
        root.right = self._delete(root.right, key)
    else:
        # Case 1: No child
        if root.left is None:
            return root.right
        elif root.right is None:
```

```
            return root.left

        # Case 2: Two children
        min_val = self._min_value(root.right)
        root.data = min_val
        root.right = self._delete(root.right, min_val)

    return root

  def _min_value(self, node):
    while node.left:
        node = node.left
    return node.data
bst = BinarySearchTree()

bst.insert(50)
bst.insert(30)
bst.insert(70)
bst.insert(20)
bst.insert(40)
bst.insert(60)
bst.insert(80)

print("Min:", bst.find_min())
print("Max:", bst.find_max())

bst.search_level(60)

print("Total Nodes:", bst.count_nodes(bst.root))
print("Left Subtree Nodes:", bst.count_left_subtree())
print("Right Subtree Nodes:", bst.count_right_subtree())
```

```
bst.delete(70)

print("Deleted 70")
```

```
on/bst.py
Min: 20
Max: 80
Element found at level 3
Total Nodes: 7
Left Subtree Nodes: 3
Right Subtree Nodes: 3
Deleted 70
PS C:\Users\DELL\Desktop\python>
```

1. Write a program to search element using Binary search

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1


arr = [10, 20, 30, 40, 50]

print("Binary Search: Index of 30 =", binary_search(arr, 30))
```

```
on/algorithms.py
Binary Search: Index of 30 = 2
```

2. Write a program to generate Pascal Triangle

```
def pascal_triangle(n):
    triangle = []
    for i in range(n):
        row = [1] * (i + 1)
        for j in range(1, i):
            row[j] = triangle[i-1][j-1] + triangle[i-1][j]
        triangle.append(row)
    return triangle

print("Pascal Triangle:")
for row in pascal_triangle(5):
    print(row)
```

```
Pascal Triangle:
[1]
[1, 1]
[1, 2, 1]
[1, 3, 3, 1]
[1, 4, 6, 4, 1]
```

3. Write a program of Tower of Hanoi for n=3

```
def tower_of_hanoi(n, source, target, auxiliary):
    if n == 1:
        print(f"Move disk 1 from {source} to {target}")
        return
    tower_of_hanoi(n-1, source, auxiliary, target)
    print(f"Move disk {n} from {source} to {target}")
    tower_of_hanoi(n-1, auxiliary, target, source)

print("Tower of Hanoi for n=3:")
```

```
tower_of_hanoi(3, 'A', 'C', 'B')
```

```
Tower of Hanoi for n=3:
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

4. Write a program to find GCD using Euclidean Algorithm

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

print("GCD of 48 and 18 =", gcd(48, 18))
```

```
GCD of 48 and 18 = 6
```

5. Write a program to sort elements in the list using Merge Sort

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        L = arr[:mid]
        R = arr[mid:]

        merge_sort(L)
        merge_sort(R)
```

```
    i = j = k = 0
    while i < len(L) and j < len(R):
        if L[i] < R[j]:
            arr[k] = L[i]
            i += 1
        else:
            arr[k] = R[j]
            j += 1
        k += 1

    while i < len(L):
        arr[k] = L[i]
        i += 1
        k += 1
    while j < len(R):
        arr[k] = R[j]
        j += 1
        k += 1

arr = [38, 27, 43, 3, 9, 82, 10]
merge_sort(arr)
print("Merge Sort:", arr)
```

```
Merge Sort: [3, 9, 10, 27, 38, 43, 82]
```

6. Write a program to sort elements in the list using Quick Sort

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr)//2]
```

```
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)

arr = [10, 7, 8, 9, 1, 5]
print("Quick Sort:", quick_sort(arr))
```

```
Quick Sort: [1, 5, 7, 8, 9, 10]
```

7. Write a program to implement Jump Game

```
def can_jump(nums):
    max_reach = 0
    for i, num in enumerate(nums):
        if i > max_reach:
            return False
        max_reach = max(max_reach, i + num)
    return True

nums = [2, 3, 1, 1, 4]
print("Jump Game possible?", can_jump(nums))
```

```
Jump Game possible? True
```

8. Write a program to find unique paths for m*n matrix

```
def unique_paths(m, n):
    dp = [[1]*n for _ in range(m)]
    for i in range(1, m):
        for j in range(1, n):
```

```
        dp[i][j] = dp[i-1][j] + dp[i][j-1]

    return dp[m-1][n-1]


print("Unique Paths in 3x3 matrix =", unique_paths(3, 3))
```

```
Unique Paths in 3x3 matrix = 6
PS C:\Users\DELL\Desktop\python>
```

```
        return None
    return self.front.data

    # 4. Reverse Queue using Stack
    def reverse(self):
        stack = []
        curr = self.front
        while curr:
            stack.append(curr.data)
            curr = curr.next

        curr = self.front
        while curr:
            curr.data = stack.pop()
            curr = curr.next

    # 5. Count occurrences
    def count_occurrences(self, key):
        count = 0
        curr = self.front
        while curr:
            if curr.data == key:
                count += 1
            curr = curr.next
        return count

    # 6. Search element
    def search(self, key):
        pos = 0
        curr = self.front
        while curr:
            if curr.data == key:
                return pos
            curr = curr.next
            pos += 1
        return -1

    # 7. Merge two queues
    def merge(self, q2):
        merged = Queue()
        curr = self.front
        while curr:
            merged.enqueue(curr.data)
            curr = curr.next

        curr = q2.front
        while curr:
            merged.enqueue(curr.data)
            curr = curr.next

        return merged

    # 8. Check identical queues
    def is_identical(self, q2):
        p = self.front
        q = q2.front
        while p and q:
            if p.data != q.data:
                return False
            p = p.next
            q = q.next
        return p is None and q is None

    # Display
    def display(self):
        curr = self.front
        while curr:
            print(curr.data, end=" -> ")
            curr = curr.next
        print("NULL")


# ================= PRIORITY QUEUE ========
class PriorityQueue:
    def __init__(self):
        self.front = None

    # 1. Enqueue with priority (lower number = higher priority
    def enqueue(self, data, priority):
        new_node = Node(data, priority)

        if self.front is None or priority < self.front.priority:
            new_node.next = self.front
            self.front = new_node
            return

        curr = self.front
        while curr.next and curr.next.priority <= priority:
            curr = curr.next

        new_node.next = curr.next
        curr.next = new_node

    # 2. Display Queue
    def display(self):
        curr = self.front
        while curr:
            print(f"[{curr.data}, p={curr.priority}]", end=" -> ")
            curr = curr.next
        print("NULL")

    # 3. Dequeue highest priority element
    def dequeue(self):
        if self.front is None:
            print("Priority Queue is Empty")
            return None
        removed = self.front.data
        self.front = self.front.next
```